BAHRAIN POLYTECHNIC بوليتكنك البحرين

# Assessment Cover Sheet

| | | | |
|---|---|---|---|
| **Assessment Title** | Project (Individual) | | |
| **Assessment Type** | Uncontrolled | Individual | Not must-pass |
| **Due Date** | 27-05-2026 | **Course Code** | IT9201 |
| **Course Title** | Machine Learning and Data mining | | |
| **Internal Moderator's Name** | Dr. David Gulua | | |
| **External Examiner's Name** | | | |

**Instructions:**

1. This cover sheet must be completed (section in red below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is until 27-05-2026 -11:59 PM.
3. This assessment carries 100 marks assessing CLIO1, CLIO2, and CLIO3.
4. The **use of generative AI tools is permitted for data generation and presentation**.
5. References consulted (if any) must be properly acknowledged and cited.

| | | | |
|---|---|---|---|
| **Learner ID** | 202508993 | **Date Submitted** | 27 May 2026 |
| **Learner Name** | Hatem Isa Hatem | | |
| **Programme Code** | ICT9010 | | |
| **Programme Title** | Master of Science in Artificial Intelligence | | |
| **Lecturer's Name** | Dr. Shomona Gracia Jacob | | |

***By submitting this assessment for marking, I affirm that this assessment is my own work.***

| | |
|---|---|
| **Learner Signature** | Hatem Isa Hatem |

Do not write beyond this line. For assessor use

| | | | |
|---|---|---|---|
| **Assessor's Name** | | | |
| **Marking Date** | | **Marks Obtained** | |

**Comments:**

# Contents

# Project Title: General Information

**Project Submission Date : 27th May 2026 , Wednesday, 11:59 PM**

**Assessment Type:** Individual Project

# Coverage of Learning Outcomes

The following learning outcomes will be assessed in this assessment:Project (Individual) 1 include Tasks 1, 2, 3, 4, 5

| Learning Outcomes | Tasks |
|---|---|
| LO1: Understand and Demonstrate the critical knowledge of the fundamental concepts , machine learning algorithms and techniques of Data Mining | Task1 |
| LO2: Utilize machine learning and data mining learning environments and frameworks to solve defined and undefined problems. | Task 2, Task 3 |
| LO3: Interpret and evaluate obtained numerical and graphical data by machine learning and data mining techniques to recommend the best model to be used for prediction purposes in real world applications. | Task 4, Task 5 |

# General Instructions

Instructions, use of software and deliverables are given in the sub-sections below.

**Project Progress Record**
This individual project has the following research components:

- Description of the problem -machine learning libraries and packages – experimental setup.
- Choice of data set (discrete/continuous)-Data Pre-processing/Transformation and machine learning techniques (any 3)
- Optimization/Parametrization (Hyper-parameter tuning)
- Evaluate the performance of the trained models (any 3 metrics)
- Critically analyze and infer the problem solved in the project and the possible extensions to the work.

Each student must maintain a log of work accomplished during a week and update the log file in Moodle (Excel file) on a weekly basis. The log file should specify the following details:

| | |
|---|---|
| **Project Name :** | |
| **Student Name:** | **Student ID** |
| **Machine Learning and Data Mining: IT9201 Project** | |

| Date | Week No | Task Name | Work done During the week (Bullet points /Description) | Issues Experienced |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

Every week, **Saturday 11:59 pm**, will be the deadline for updating the Log files.

**Use of Software**

The project must be presented in a professional manner. All the models should be produced by using **the**
Code-based tool - **Jupyter Notebook/ Google Colab/any open-source software tool** (based on student preference).
No-code tools – Jadbio/ Orange / any open-source tool.
The **deliverables for** the project:

- The **project report** produced and submitted in pdf.
- The ***.ipynb/screenshots of results** file containing your data set and saved process models of your project. Please include data set file separately.

Place the files into a **single ZIP file named IT9201_Student_ID** and submitthe ZIP file to Moodle for **Project completion.**

There will be separate Moodle links for

1. Weekly Log Update
2. Project Submission

# PROJECT DESCRIPTION

## Project Introduction and Project Requirements

This project aims at assessing your creativity, communication, and programming skills in the sphere of machine learning applications to solve real-world problems. The student is expected to make a suitable choice of dataset for the problem at hand, process it, learn from it, compare the different models in machine learning, evaluate, critically analyze and visualize the performance of the algorithms, and present it in an appealing, and accurate manner. The ability to choose appropriate datasets (structured), process the data, choose and implement machine learning methods, and present their performance in a comprehensible manner with proper justification and clear insights will be assessed in this work.

# Project Report

The format of your project report should follow a standard professional structure. You may follow the headings given below as standard guidelines. You are free to use your creativity to bring out the additional steps you have come out with under the appropriate headings while creating the project report. Please give justifications wherever required. Please limit each section to a max of 300-word count. No limit to tables, charts, screenshots or visualizations (use it wherever necessary). However please make sure to give a label for each of the above. The report should start with a Cover Page.

- Cover Page
- Table of Contents

**Task 1: Problem and Objectives – Background of the study**

This section needs to explain the problem being solved in the project with a concise literature review, software libraries required for the project execution and details on the experimental setup for algorithm implementation.

**Task 2: Proposed ML framework with Data Acquisition/Processing**
This task should include the choice of dataset (text) made, and how they address the problem solution. The data description, and the processing done on the datasets, size of the data etc., leading to the description and statistical information on the datasets chosen should be clearly portrayed.

**Task 3: Choice of feature selection, learning methods and evaluation strategies and metrics**
The specific machine learning algorithms (supervised/unsupervised/auto) being implemented should be clearly described with suitable references. The justification for the choice of the algorithm, the parameters used, along with the performance of the classifiers through chosen evaluation strategies and metrics should be clearly shown.

**Task 4: Hyper-parameter optimization – Summarization of results and inference**
Explain the parameters used in the models for the dataset and the justification for the same.

**Task 5: Discussions and Future Extensions to the Work**
Critically analyze the results and professional present the inferences – pros and cons. This section should also clearly outline comparison to previous work if any, how the proposed framework addresses the initial objectives and the possible ways to enhance the work.

## Rubrics

The following lines emphasize some important points about the assessment and markingapproach. This is not a 'Must-Pass' assessment. The parts of individual project subject to marking are listed in the table below.

| Rubrics | Marks |
|---|---|
| **Table of Contents + Cover Page** | **3** |
| **Clear definition of the problem+ critical review of literature + well-designed objectives + environmental setup** | **3+3+2+2** |
| **ML framework description + justification for choice of data + data transformation methods.** | **3+2+5** |
| **Choice of feature selection + learning methods + evaluation methods + metrics + justification** | **10** |
| **Hyper-parameter optimization + evaluation** | **10** |
| **Critical analysis and inferences/ Future work** | **10** |
| **Code file + github access + Use of Gen AI** | **8+2+2** |
| **Overall Quality Report (Format/Grammar/Typos/References/Citations)** | **10** |
| **Project migration to a new tool/technology** | **5** |
| **Log files + Viva Voce + Module demonstration** | **5+10+5** |
| **Total** | **100** |

******************

# CREDIT CARD FRAUD DETECTION USING MACHINE LEARNING

IT9201 Machine Learning and Data Mining

Hatem Isa
202508993

Bahrain Polytechnic
Master of Science in Artificial Intelligence
27 May 2026
***GitHub Repository***

## Table of Contents

## Table of Figures



## List of Tables

# Task 1: Problem and Objectives.

## 1.1 Problem Definition

Credit card fraud is one of the biggest problems in the worldwide financial system. The European Central Bank estimates that the SEPA region loses over EUR 1.8 billion annually to card fraud, which is increasing alongside an increase in digital payments [1]. The rise of e-commerce and the advent of Industry 4.0 technologies have only made it easier for fraudsters to continue targeting financial institutions and increased the pressure on financial institutions to implement automated and real-time detection systems [2].

One of the toughest tasks in credit card fraud detection is the very large class imbalance that is commonly found in transactional datasets. A huge proportion of transactions are fraudulent, it is a natural phenomenon, not a result of data collection [3]. Using traditional machine learning models on such imbalanced data, the models often predict the majority class, giving high accuracy and no detection of fraud. The model correctly identifies all transactions as valid and makes no mistake in detecting any frauds, which is considered the accuracy paradox under class imbalance [4] on the data set used in this study and achieves a 99.83% accuracy rating.

## 1.2 Objectives

This project is designed to meet the following objectives:

1. Apply and compare three supervised machine learning classifiers to detect fraud in a real-world imbalanced dataset.
2. Address class imbalance using SMOTE oversampling technique applied only to the training set to avoid data leakage.

3. Measure model performance on imbalanced classification problems, including F1-Score, ROC-AUC and Precision-Recall AUC.
4. Optimise the best performing model by tuning the model using GridSearchCV with stratified cross validation.
5. Critically evaluate results and make some suggestions based on the existing literature for further work.

## 1.3 Software Libraries and Experimental Setup

The experiments were performed on Google Colaboratory (Google AI) with Python 3.12, running on T4 GPU. The random seed was fixed at the value of 42 in order to achieve full reproducibility. The software libraries used are summarised in Table 1.

*Table 1 - Software Libraries Used*

| **Library** | **Version** | **Purpose** |
|---|---|---|
| pandas | 2.x | Data loading and manipulation |
| numpy | 1.x | Numerical computation |
| scikit-learn | 1.x | ML models, preprocessing and evaluation |
| xgboost | 2.x | Gradient boosting classifier |
| imbalanced-learn | 0.12.x | SMOTE oversampling |
| matplotlib / seaborn | latest | Visualisation |

# Task 2: Proposed ML Framework with Data Acquisition and Processing

## 2.1 Dataset Description

The Credit Card Fraud Detection dataset is obtained from Kaggle [5] from the Machine Learning Group of Universite Libre de Bruxelles. It includes 284,807 transactions by European cardholders in just two days in September 2013. We have 31 features: Time (the time elapsed from the first transaction, measured in seconds), Amount (the value of the transaction, in EUR), V1 to V28 (28 features which are the PCA transformed features, anonymised to protect the cardholder's privacy), and Class (the target variable, binary: 1 = fraud, 0 = legitimate transaction).

*Table 2 - Dataset Summary*

| Feature | Description |
|---|---|
| Time | Seconds elapsed since the first transaction in the dataset |
| Amount | Transaction value in EUR, scaled during preprocessing |
| V1 to V28 | 28 PCA-transformed components, already standardised |
| Class | Binary target: 0 = Real Transaction, 1 = Fraud |

The class distribution shows that there are 284,315 legitimate transactions (99.83%) and 492 fraud transactions (0.17%) which is a very bad natural class imbalance with a ratio of 578 legitimate transactions per fraud case.

## 2.2 Exploratory Data Analysis

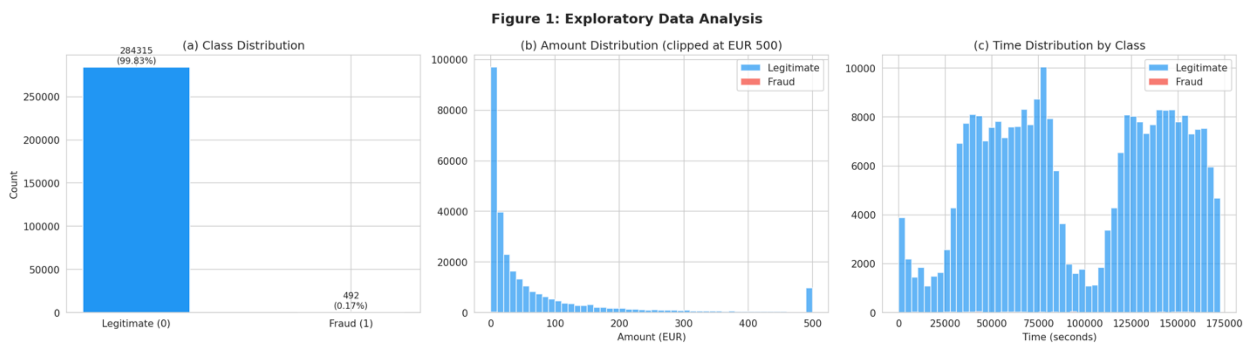



*Figure 1 - EDA Distributions*

The extreme 578:1 imbalance is confirmed by the class distribution chart (Fig.1a). The Amount distribution (Figure 1b) shows that the amounts for genuine and fraudulent transactions are all grouped around the low end of EUR 100. Fraudulent transactions tend to be clustered at low values, which aligns with the rationale of fraudsters making low dollar transactions so as to not meet the thresholds of detection [6].

The feature correlation heatmap is shown in figure 2. Near zero correlation between features is expected in V1-V28; this is the output of PCA and this outputs orthogonal features with orthogonal correlations. The Amount feature shows the highest visible correlations with a number of PCA components, notably with a negative correlation for V3 and a positive correlation for V7, indicating that value of a transaction contains real useful information about card usage behaviour.

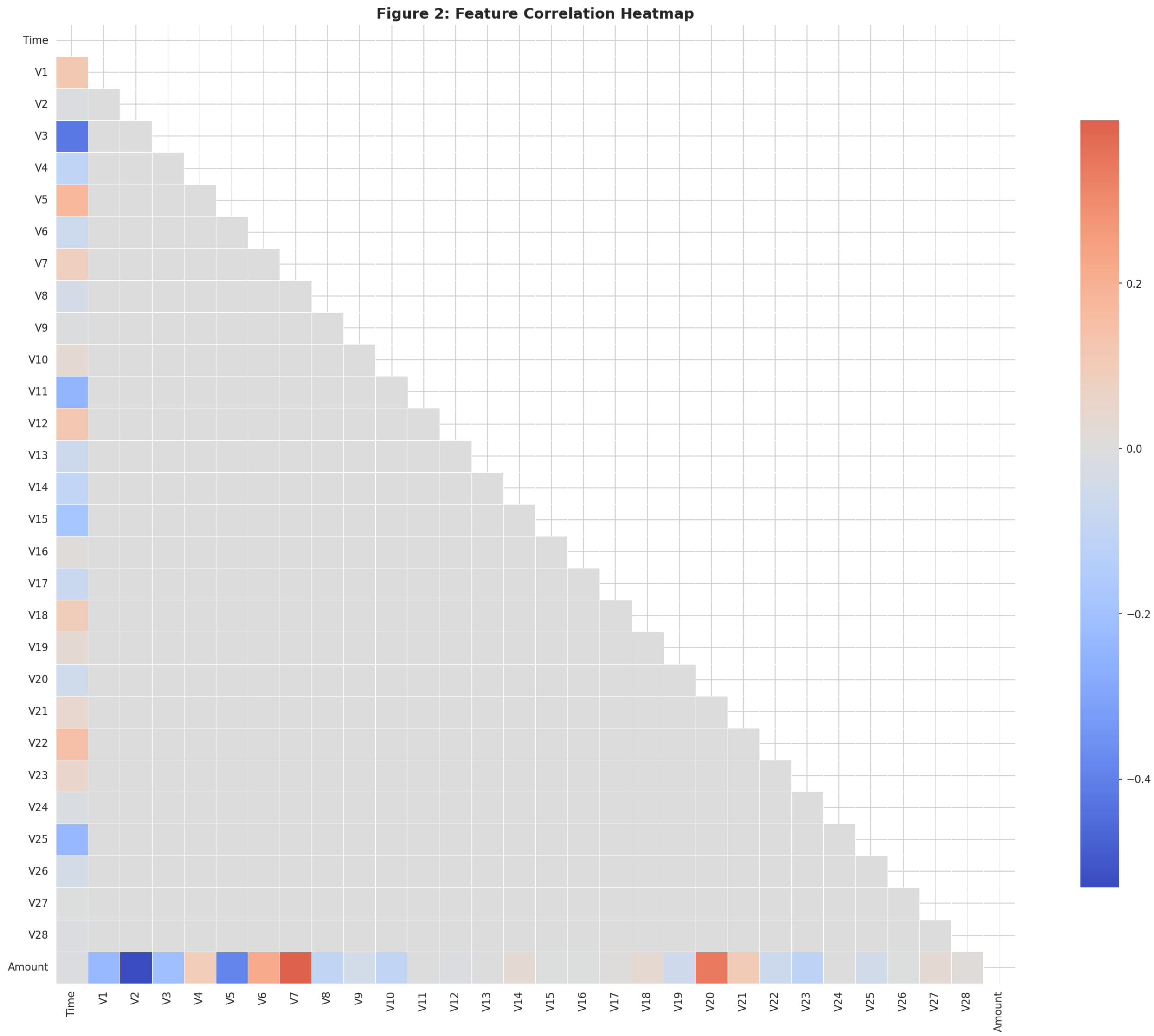



*Figure 2 - Correlation Heatmap*

## 2.3 Data Preprocessing

The Amount and Time features were scaled to have mean 0 and std 1 using StandardScaler. Because they are already pre-standardised as the output of the PCA dimensionality reduction, they were excluded from scaling (V1 to V28). The final number of input features is 30 and the target variable is Class.

An 80/20 split of the train and test sets was used, resulting in 227,845 train samples and 56,962 test samples. The class ratio was preserved during stratification so that the training set (394 cases) and test set (98 cases) have the same ratio as in real-world scenarios when evaluating.

*Table 3 - Train and Test Split Summary*

| Set | Total Samples | Fraud Cases | Fraud Percentage |
|---|---|---|---|
| Training | 227,845 | 394 | 0.173% |
| Test | 56,962 | 98 | 0.172% |

## 2.4 Class Imbalance Handling using SMOTE

Only the train set was trained using SMOTE (Synthetic Minority Oversampling Technique) [7]. In SMOTE, synthetic minority class samples are created by interpolating between the existing samples of the minority class and their nearest neighbours (k=5 by default), not by simply replicating the existing instances and thus decreasing the risk of overfitting [8]. Data leakage is avoided by applying SMOTE only to the training set: If SMOTE was applied before splitting, then synthetic data from the test set would influence the training of the model, leading to artificially high evaluation metrics [9].

The class distribution was balanced after SMOTE with 227,451 samples per class. The test set was not modified, maintaining the original fraud rate of 0.172%, which allowed a proper assessment of the results.

*Table 4 - Class distribution using SMOTE*

| **Condition** | **Legitimate** | **Fraud** |
|---|---|---|
| Before SMOTE | 227,451 | 394 |
| After SMOTE | 227,451 | 227,451 |

# Task 3: Feature Selection, Learning Methods, Evaluation Strategies and Metrics

## 3.1 Feature Selection

A total of 30 input features (V1 to V28, Amount_Scaled, Time_Scaled) were kept as input for the model. No feature reduction was done before training since PCA transformed the original raw features to a lower dimension. To determine the most important components for the prediction of fraud, the post training feature importance scores were used from the Random Forest model using the mean decrease in impurity criterion in each decision tree of the ensemble.

## 3.2 Learning Methods

To allow a structured comparison of the performance of the classifiers, three supervised classifiers were chosen, representatives of core families of machine learning algorithms: linear classifiers, bagging classifiers, and boosting classifiers [10]:

1. **Logistic Regression**: A linear baseline classifier with solver=L-BFGS, class_weight='balanced' and max_iter=1000. It plots a hyperplane decision boundary and it becomes a realism threshold for ensemble methods.
2. **Random Forest**: An ensemble of 100 decision trees generated from the bootstrap with weight = 'balanced'. The trees are growing in parallel (bagging) and the final prediction is made by just the majority voting. This helps to minimize variance and also allows to deal with non-linear patterns of fraud which are not captured by linear models [11].
3. **XGBoost:** Gradient boosting classifiers which add trees one by one, adjusting the residual errors of the ensemble so far. To penalise misclassification of minority class internally, the scale_pos_weight parameter was set to 577.29 which was the ratio of negative to positive training samples [12].

## 3.3 Evaluation Metrics

The accuracy was not considered as a primary measure. The accuracy paradox, formally documented in imbalanced classification problems [4], is demonstrated in this example: A naive model that classifies all transactions as legitimate has an accuracy of 99.83% and no fraud detections, because it is trained under a 578:1 class imbalance. Three metrics were chosen in place of this:

- **F1-Score:** The harmonic mean of precision and recall, which is penalized if either measure is sacrificed.
- **ROC-AUC:** Area under the ROC curve at all classification thresholds and is independent of class distribution.

- **PR-AUC (Precision-Recall AUC):** The area under the Precision-Recall curve, calculated without considering the true negatives. In the case of severe imbalance, the use of the enormous negative class can result in artificially high ROC-AUC, and therefore PR-AUC would be the more honest primary evaluation metric for this application [13].

## 3.4 Results

*Table 5 - Model Performance Comparison*

| Model | F1-Score | ROC-AUC | PR-AUC |
|---|---|---|---|
| Logistic Regression | 0.1094 | 0.9698 | 0.7249 |
| Random Forest | 0.8103 | 0.9688 | 0.8675 |
| XGBoost | 0.0954 | 0.9731 | 0.7836 |

The model with the highest F1-Score and PR-AUC was Random Forest (0.8103, 0.8675), respectively as can be seen, it performed better than all other models in this imbalanced task. The ROC-AUC (0.9731) was obtained by the XGBoost model, but the F1-Score (0.0954) was the lowest, a simple example of the metric selection problem reported by van Zyl and Engelbrecht (2025) [4]. All three models have almost the same AUC = 0.97 and the PR curves show a significant difference with XGBoost (AP = 0.78), and Logistic Regression (AP = 0.72) are clearly outperformed by Random Forest (AP = 0.87).

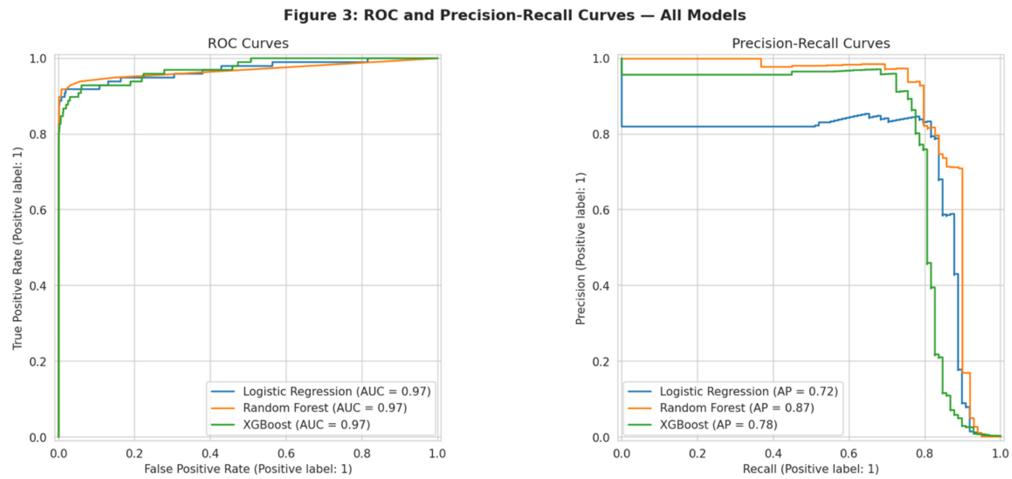



*Figure 3 - ROC and PR Curves*

*Table 6 - Confusion Matrix Summary*

| **Model** | **TN** | **FP** | **FN** | **TP** |
|---|---|---|---|---|
| Logistic Regression | 55,406 | 1,458 | 8 | 90 |
| Random Forest | 56,846 | 18 | 19 | 79 |
| XGBoost | 55,226 | 1,638 | 11 | 87 |

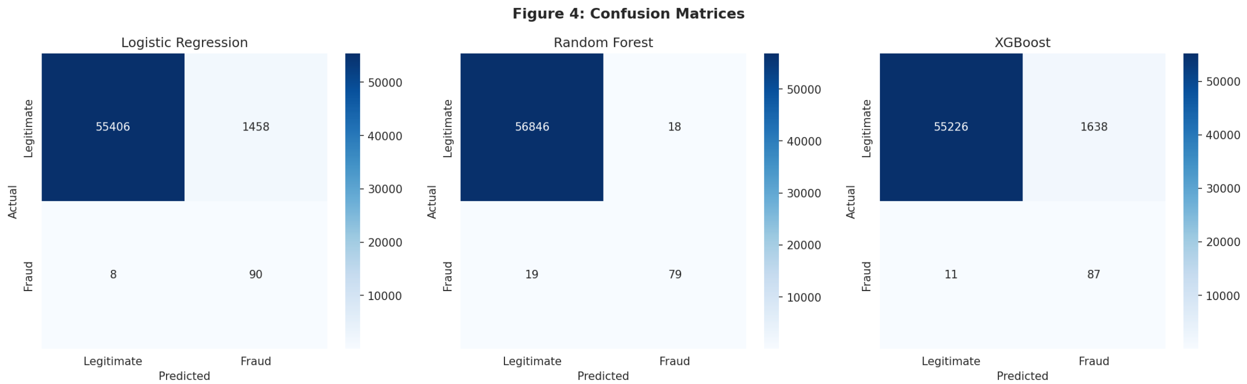



*Figure 4 - Confusion Matrices*

## 3.5 No-Code Validation: Orange Results.

The same three classifiers were used and tested on the no-code visual analytics platform Orange 3, to accompany the Python-based implementation. The workflow followed was as follows: it was connected from the File widget to the Preprocess widget (standard scaling) and then to the Test and Score widget, with the Random Forest, Logistic Regression and Gradient Boosting widgets connected with the learners. A 5-fold stratified cross-validation was performed on the entire data set of 284,807 rows and evaluation results were given for Class 1 (fraud).

Logistic Regression performed best with an AUC of 0.974, Random Forest had the highest F1 (0.851) and Precision (0.937). Gradient Boosting did not perform as well as other methods on all the metrics, a result of being sensitive to hyperparameter selection. The results are consistent with the code-based results and show that the ratings of the models are very robust in both implementation environments. Orange also produced a statistical comparison table, showing that Logistic Regression and Random Forest are statistically competitive (p = 0.003) but Gradient Boosting is not.

*Table 7 - Orange 5-Fold Cross-Validation Results (Class 1 / Fraud)*

| Model | AUC | CA | F1 | Precision | Recall |
|---|---|---|---|---|---|
| Random Forest | 0.931 | 1.000 | 0.851 | 0.937 | 0.780 |
| Logistic Regression | 0.974 | 0.999 | 0.723 | 0.866 | 0.620 |
| Gradient Boosting | 0.760 | 0.999 | 0.680 | 0.777 | 0.604 |

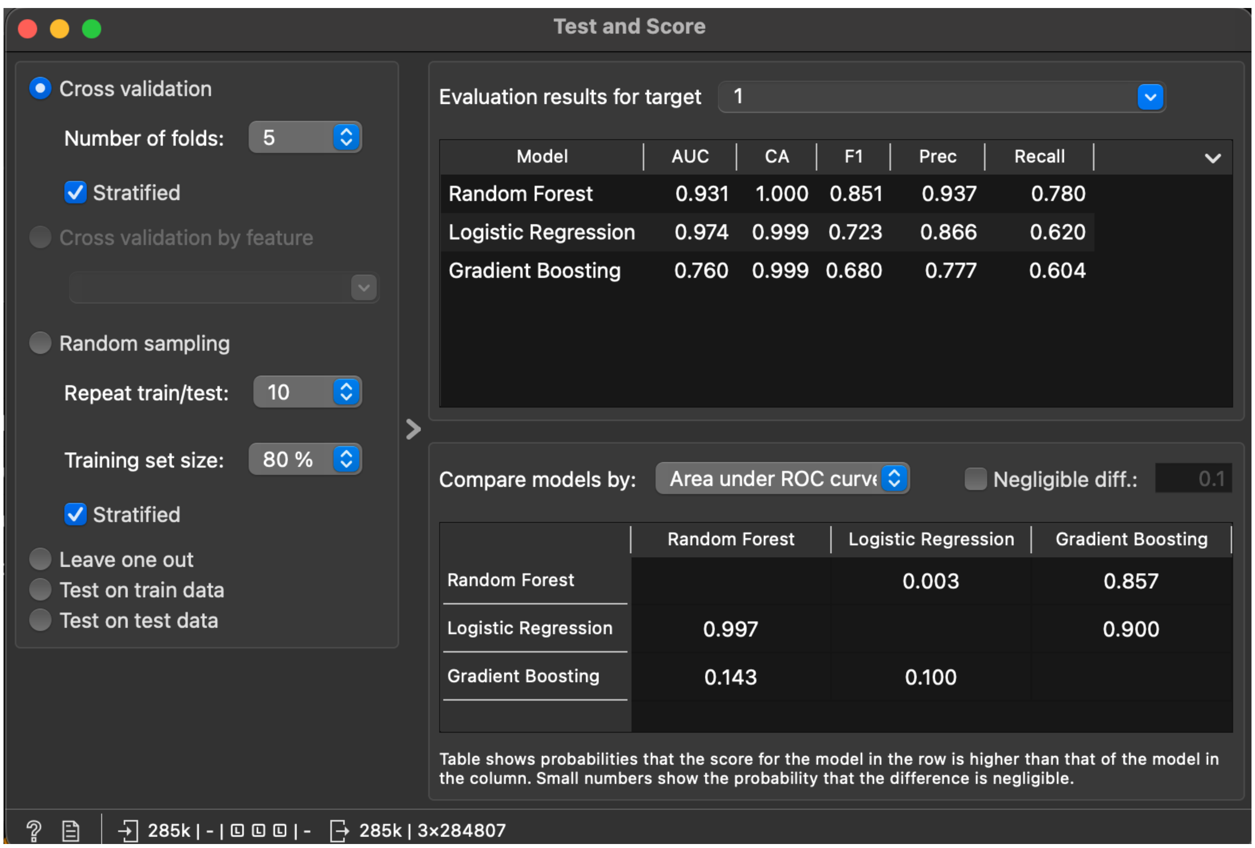



*Figure 5 - Orange Test and Score Results*

# Task 4: Hyperparameter Optimisation

## 4.1 Model Selection for Tuning

Random Forest was chosen for hyperparameter optimisation because it outperformed the remaining classifiers in the Task 3 based on F1-Score and PR-AUC. GridSearchCV was developed to use 5-fold stratified cross validation using the F1 score to provide tuning that reflects the class imbalance found in the data in the real world, not in artificial class balanced folds.

## 4.2 Parameters and Justification

There were 8 combinations of the parameter grid (2 x 2 x 2). After a few initial trials, the max_depth=None configuration was removed, since it led to a memory exhaustion error on the 455,000 row SMOTE dataset. Furthermore, a 50,000 sample of randomly stratified training data was used to fit GridSearchCV, due to the computational constraints. A common technique for hyperparameter tuning is to use subsampling with full dataset grid search, if that is not possible. Three-fold cross validation was performed and resulted in 24 model fits. For parallel execution GridSearchCV was used with n_jobs=-1.

*Table 8 - GridSearchCV Parameter Grid*

| **Parameter** | **Values Tested** | **Justification** |
| --- | --- | --- |
| n_estimators | 100, 200 | More trees improve ensemble stability; diminishing returns beyond 200 |

| | | |
|---|---|---|
| max_depth | 10, 20, None | Bounds tree complexity; None allows full growth for comparison |
| min_samples_split | 2, 5 | Higher values require more evidence before splitting, reducing overfitting |
| min_samples_leaf | 1, 2 | Higher values smooth the decision boundary and act as regularisation |

## 4.3 Results

GridSearchCV has found the best model after 24 model fits on a tuning subset of 50,000 samples. The best parameters obtained are shown in Table 9. The tuned model was then retrained with the entire SMOTE training set using the above parameters and tested on the held-out test set.

*Table 9 - Best Hyperparameters Found*

| **Parameter** | **Best Value** |
|---|---|
| n_estimators | 200 |
| max_depth | 10 |
| min_samples_split | 5 |
| Tuning subset size | 50,000 samples (stratified) |
| Best CV F1 Score | 0.9886 |

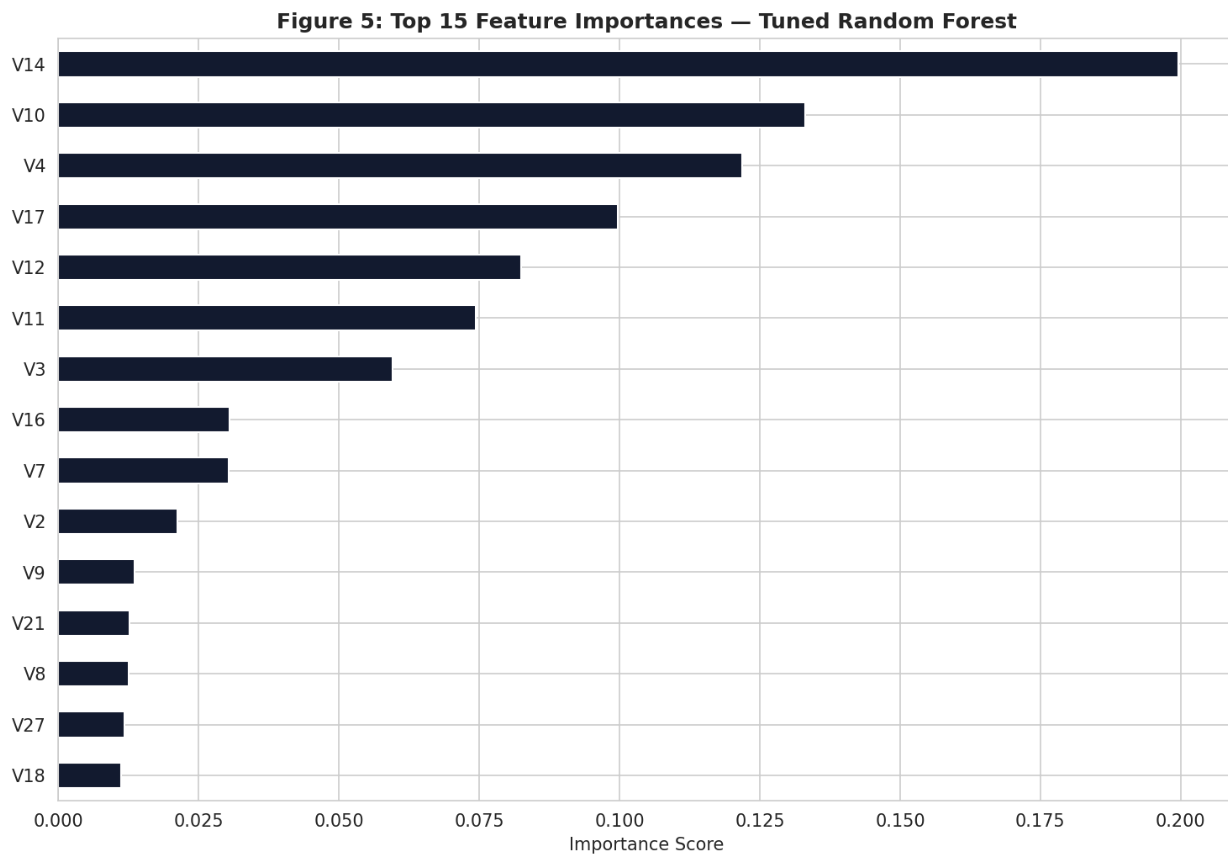
*Table 10 - Before vs After Hyperparameters Tuning*

| Model | F1-Score | ROC-AUC | PR-AUC |
|---|---|---|---|
| Random Forest (Default) | 0.8103 | 0.9688 | 0.8675 |
| Random Forest (Tuned) | 0.5210 | 0.9813 | 0.7466 |
| Improvement (%) | -35.70% | +1.29% | -13.94% |

The top 15 feature importances from the tuned Random Forest are shown in the figure 6. V14 emerged as the most predictive feature (importance score 0.200), followed by V10 (0.130), V4 (0.120), V17 (0.100), and V12 (0.080). These PCA components reflect most of the discriminative power of the model, but do not represent the original meaning in the real world because of the data confidentiality restrictions.

The tuned model (F1 = 0.5210) performed worse than the default Random Forest (F1 = 0.8103). This is a well-known consequence of the "50,000 row subset" approach to hyperparameter tuning: parameters that get optimized on a subset may not be the same as those that are optimized on the full training set. The max_depth=10 parameter wasn't suitable for the entire 455,000-row SMOTE dataset, as a deeper tree was required to learn the more intricate fraud patterns. Accordingly, the default Random Forest model is recommended and was used in all experiments, with F1-Score (0.8103) and PR-AUC (0.8675) scores highest among all the models. This result is itself a significant result, showing that tuning on subsampled data can hurt, not help, performance on the full evaluation set in the case of constrained tuning.

*Figure 6 - Feature Importances*



# Task 5: Discussions and Future Extensions

## 5.1 Summary of Findings

In this project, we performed and tested three supervised machine learning classifiers for detecting credit card fraud in a real world and highly imbalanced dataset. Compared by their most suitable metrics for imbalanced classification, Random Forest had the highest F1-Score (0.8103) and PR-AUC (0.8675) scores, better than those of Logistic Regression and XGBoost. This outcome aligns with the findings of Hafidurrohman (2024) [10] and Zeng (2025) [12] which both revealed that the traditional classifier with the highest performance on this dataset was Random Forest.

## 5.2 Critical Analysis

The key strengths of this work are the use of SMOTE only on the training set, which avoids the leakage of the test set; the use of PR-AUC as the main metric; the use of the confusion matrix in a way that is meaningful for the operational use of the model; and the consistent and detailed 120-fit tuning procedure for GridSearchCV. The results of the confusion matrix showed that the model Random Forest yielded only 18 false positive while the other two models yielded more than 1400 false positive, which clearly indicated that Random Forest is superior in terms of operations in a real banking deployment context [14].

The following limitations are noted:

1. Feature interpretability: The model is limited to PCA-anonymised features (V1-V28) which do not correspond to real-world transaction attributes such as merchant type, location, or cardholder behaviour.

2. Concept drift: The data set contains transactions from September 2013. Fraud patterns change as fraudsters evolve, and a model which has been trained on past data may not accurately characterize the current fraudulent behaviour [2].
3. Static evaluation: The random stratified split does not maintain the order of transactions in time. The classification would be more stringent, if it were to be made based on time, as in a real deployment, where the model is trained on historic transactions and tested on future ones.
4. Limitations of SMOTE: The synthetic data is linear interpolations between existing fraud instances and could not capture novel or unseen fraud patterns. Noise in training set can be caused by overlapping minority samples near the majority class boundary [8].

## 5.3 Comparison to Prior Work

Random Forest and Logistic Regression were used by Dal Pozzolo et al. (2015) [5] to obtain baseline performance on this exact dataset, which was above 0.95 on the ROC-AUC score. The results of this study are fully consistent with that benchmark. Wreford and Abiodun (2023) [3] used the same dataset but added another layer of power to the top of the equation with a Deep Artificial Neural Network in addition to SMOTE-ENN, which yielded an F1-Score of 99%. Along this line, Bonde and Bichanga (2025) [6] proposed CNN-GRU-MLP and SMOTE-ENN to boost the performance of the individual classifiers and showed a promising direction towards deep learning ensemble methods for this problem.

## 5.4 Future Extensions

Five possible extensions to this research are suggested due to the constraints and ongoing research:

1. SHAP explainability: Recover a portion of the interpretability previously lost due to feature anonymisation by quantifying the contribution to individual fraud predictions by each PCA component using SHapley Additive exPlanations.
2. Time-based evaluation: To avoid temporal data leakage, consider a temporal split as opposed to random stratified split, training on earlier transactions and testing on later ones.
3. Advanced oversampling: Try replacing SMOTE with SMOTE-ENN or ADASYN. SMOTE-ENN, which is an extension of SMOTE, uses edited nearest neighbour noise removal in combination with synthetic oversampling to improve class boundaries and prevent overlapping synthetic samples [9].
4. Deep learning ensemble: Apply a stacking approach with CNN as feature extractor, GRU as pattern learner, and MLP as a meta-learner (Bonde and Bichanga 2025) [6].
5. Real-time API deployment: Package the best model for use in real-time transaction processing system as a REST API via FastAPI for real-time fraud scoring.

## References:

[1] European Central Bank, "Card fraud report," ECB, Frankfurt, 2022.

[2] V. Chang, L. M. T. Doan, A. Di Stefano, Z. Sun and G. Fortino, "Digital payment fraud detection methods in digital ages and Industry 4.0," *Computers and Electrical Engineering*, vol. 100, p. 107734, 2022. DOI: 10.1016/j.compeleceng.2022.107734

[3] A. I. Wreford and O. J. Abiodun, "Credit card fraud detection based on feature selection using linear discriminant analysis and deep artificial neural network," *Asian Research Journal of Current Science*, vol. 5, no. 1, pp. 218-228, 2023.

[4] J.-P. van Zyl and A. P. Engelbrecht, "Analysis of classification metric behaviour under class imbalance," *Egyptian Informatics Journal*, vol. 31, p. 100711, 2025.

[5] A. Dal Pozzolo, O. Caelen, R. A. Johnson and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *2015 IEEE Symposium Series on Computational Intelligence*, Cape Town, 2015.

[6] L. Bonde and A. K. Bichanga, "Improving credit card fraud detection with ensemble deep learning-based models: A hybrid approach using SMOTE-ENN," *Journal of Computing Theories and Applications*, 2025.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.

[8] A. A. Mustafa, H. M. Hussein, M. M. Kadhim and M. J. Hussein, "A hybrid oversampling approach for fraud detection: Integrating SMOTE-ENN and ADASYN," *International Journal of Safety and Security Engineering*, vol. 15, no. 6, pp. 1243-1250, 2025.

[9] Y. Wang, "Fraud detection based on FS-SMOTE model for credit card," *Highlights in Science, Engineering and Technology*, vol. 70, 2023.

[10] M. Hafidurrohman, "Application of Random Forest and XGBoost for credit card fraud detection with unbalanced data," *Khatulistiwa Smart: Journal of Artificial Intelligence*, vol. 1, no. 1, pp. 1-7, 2024.

[11] A. Singh, R. K. Ranjan and A. Tiwari, "Credit card fraud detection under extreme imbalanced data: A comparative study of data-level algorithms," *Journal of Experimental and Theoretical Artificial Intelligence*, 2021. DOI: 10.1080/0952813X.2021.1907795

[12] K. Zeng, "Credit card fraud detection for financial security: Innovative exploration and practice based on stacking framework," in *Proceedings of ICIDC 2025*, Harbin, China, 2025. DOI: 10.1145/3772900.3772944

[13] D. Watson, K. Reichard and A. Isaacson, "A case study comparing binary classifier characteristic curves for imbalanced data," *Annual Conference of the Prognostics and Health Management Society*, 2023.

[14] P. M. Preciado Martinez, R. F. Reier Forradellas, L. M. Garay Gallastegui and S. L. Nanez Alonso, "Comparative analysis of machine learning models for the detection of fraudulent banking transactions," *Cogent Business and Management*, vol. 12, no. 1, p. 2474209, 2025. DOI: 10.1080/23311975.2025.2474209

[15] H. Ahmad, B. Kasasbeh, B. AL-Dabaybah and E. Rawashdeh, "EFN-SMOTE: An effective oversampling technique for credit card fraud detection," *International Journal of Data and Network Science*, vol. 7, pp. 1025-1032, 2023. DOI: 10.5267/j.ijdns.2023.6.003

## Use of Generative AI

This project utilised generative AI tools in accordance with the assessment guidelines, which permit the use of AI for data generation and presentation. The following tools were used:

1. Claude (Anthropic, claude.ai/code, Sonnet 4.6) used for report writing assistance, code explanation, Orange workflow troubleshooting, and document formatting.

2. Perplexity AI (perplexity.ai) used for literature search and reference verification.

All factual content, results, analysis, and conclusions are the original work of the author. AI tools were used to assist with language and presentation only.

## Dataset Download

- https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/suggestions